FastAPI and MySQL Connection Part 2

We are using the To-do list application from FastAPI and MySQL Connection Part 1 for this activity. So if you haven't finished it yet, now is the perfect time to complete it before moving on!

In this activity, we will practice using aggregation functions and joining tables in FastAPI.

1. Create a new table for User and update the Todo table - models.py

First, let's add a User table structure that connects to the Todo table through the user_id column.

```
class Todo(Base):  13 usages
    __tablename__ = "todo"
    id = Column(Integer, primary_key=True, index=True)
    task_body = Column(String(500))
    due_day = Column(Integer)
    due_month = Column(String(10))
    due_year = Column(Integer)
    user_id = Column(Integer, ForeignKey("user.id"))

    user = relationship( argument: "User", back_populates="todos")

class User(Base):  6 usages
    __tablename__ = "user"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(100))
    age = Column(Integer)
    gender = Column(String(10))

    todos = relationship( argument: "Todo", back_populates="user")
```

Here is a general overview:

- In the Todo table, the user_id column is a foreign key that references the id column in the User table.
- The user = relationship("User", back_populates="todos") in the Todo class allows us to access the user who owns a task.
- The todos = relationship("Todo", back_populates="user") in the User class allows us to access all tasks that belong to a user.
- back_populates makes sure that both tables are connected properly, so when we access User.todos, we get all the tasks of that user, and when we access Todo.user, we get the owner of the task.

2. Create a new Pydantic model for User and update for Todo - main.py

Now, let's also add a Pydantic model to represent the user table, and add user_id in the todo model.

```
class TodoBase(BaseModel):
    id: int
    task_body: str
    due_day: int
    due_month: str
    due_year: int
    user_id: int

class UserBase(BaseModel):
    id: int
    name: str
    age: int
    gender: str
```

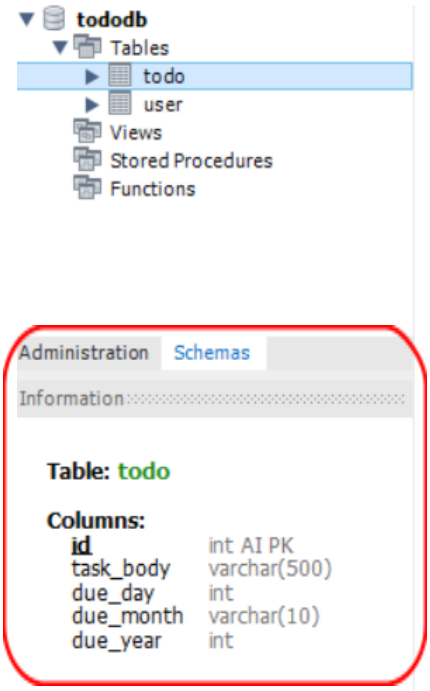
Since we've added a new table, why don't we add some endpoints for it!

Here are two endpoints—one for getting all user data and another for creating a new user.

```python
@app.get( path: "/users/", status_code=status.HTTP_200_OK, tags=["User"])
async def get_users(db: db_dependency):
    return db.query(models.User).all()

@app.post( path: "/users/", status_code=status.HTTP_201_CREATED, tags=["User"])
async def create_user(user: UserBase, db: db_dependency):
    db_user = models.User(**user.model_dump())
    db.add(db_user)
    db.commit()
    return {"detail": "User added successfully"}
```

3. Run the FastAPI application

Now, run uvicorn main:app --reload in the terminal and then go to MySQL workbench. (Don't forget to refresh the schema section!)



Notice that the user_id column we added isn't in the todo table. What happened?

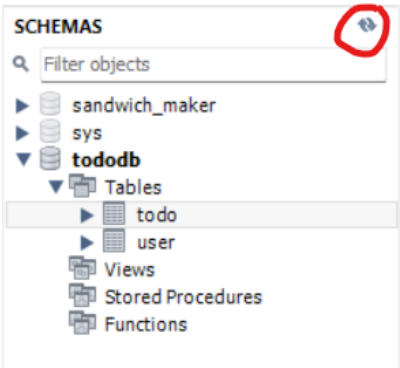
When using SQLAlchemy with FastAPI, adding a new column to an existing table in MySQL is not automatically reflected. Since the table already exists, the changes won't apply automatically, and you'll need to update it manually.

Here, we are going to simply drop the table and re-create it.

First, stop the FastAPI server (you can use Ctrl+C in the terminal), then run the following command in MySQL Workbench:

```
DROP TABLE tododb.todo;
```

Then, run the server again, and now you will see the user_id column. (You will need to refresh it again!)



SUBMISSION 1)

Take a screenshot of the updated Todo table schema (3 pts)

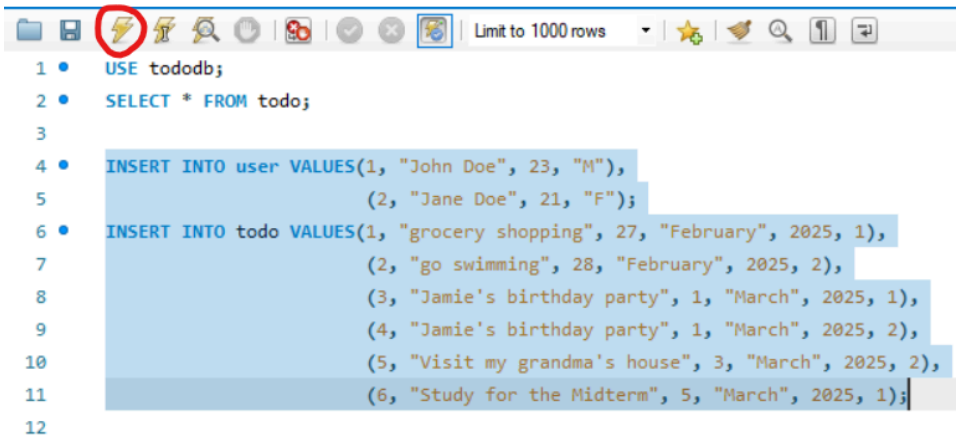
4. Insert example data into both tables

In MySQL Workbench, execute the queries below.

```
INSERT INTO tododb.user VALUES
(1, "John Doe", 23, "M"),
(2, "Jane Doe", 21, "F");

INSERT INTO tododb.todo VALUES
(1, "grocery shopping", 27, "February", 2025, 1),
(2, "go swimming", 28, "February", 2025, 2),
(3, "Jamie's birthday party", 1, "March", 2025, 1),
(4, "Jamie's birthday party", 1, "March", 2025, 2),
(5, "Visit my grandma's house", 3, "March", 2025, 2),
```

(6, "Study for the Midterm", 5, "March", 2025, 1);

When you run the queries, **make sure to select the code** you want to run and click the first lightning bolt button. (If you don't select anything, it will execute all the statements in the SQL file!)



Or, you can run each line by placing your cursor anywhere inside the statement and clicking the second lightning bolt button, or use the shortcut Ctrl+Enter.

SUBMISSION 2)

Take a screenshot of the results from the SELECT statement for both the todo and user tables. (2 pts)

5. Aggregation functions

Now we want to see how many users are there!

Make sure to add from sqlalchemy import func at the top of your main.py to use aggregation functions like func.count()

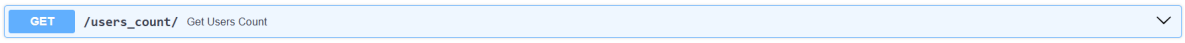
```python
from fastapi import FastAPI, HTTPException, Depends, status
from pydantic import BaseModel
from typing import Annotated
import models
from database import engine, SessionLocal, get_db
from sqlalchemy.orm import Session
from sqlalchemy import func
```

The function below defines an endpoint GET /users_count/ in FastAPI to count the number of users in the database.

- db.query(func.count(models.User.id)) creates a query that counts the number of entries in the User table.
- .scalar() extracts the single numerical value from the query result.
- return {"user_count": count_result}: The function returns a dictionary containing the count of users, which will be converted to JSON by FastAPI.

```python
@app.get( path: "/users_count/", status_code=status.HTTP_200_OK, tags=["User"])
async def get_users_count(db:db_dependency):
    count_result = db.query(func.count(models.User.id)).scalar()
    return {"user_count": count_result}
```

Now, you will see the following endpoint in the interactive API documentation.



SUBMISSION 3)

How many users are there? Use the GET /users_count/ endpoint and take a screenshot of the request body in the interactive API documentation. (3pts)

6. Joining tables

Did you know you can join tables using FastAPI? Here's an example!

We'll retrieve data for a specific user based on a given month.

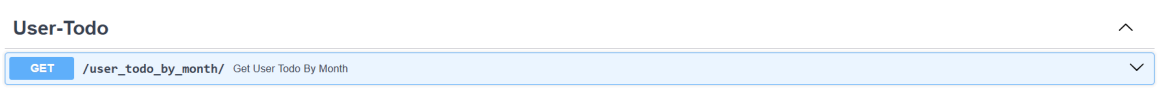
```
@app.get( path: "/user_todo_by_month/", status_code=status.HTTP_200_OK, tags=["User-Todo"])
async def get_user_todo_by_month(user_name: str, month: str, db: db_dependency):
    db_user_todos = (
        db.query(models.Todo)
        .join(models.User)
        .filter( *criterion: models.User.name == user_name, models.Todo.due_month == month)
        .all()
    )

    return db_user_todos
```

In SQLAlchemy,

- db.query(models.Todo) allows us to select all columns from the Todo table.
- .join(models.User) links Todo with User based on the relationship defined in the models.
- .filter(models.User.name == user_name, models.Todo.due_month == month) ensures that we only get tasks for the user user_name in month.
- Equivalent SQL Query:

```
SELECT todo.*
FROM todo
JOIN user ON user.id = todo.user_id
WHERE user.name = user_name AND todo.due_month = month;
```

Now, you will see the following endpoint in the interactive API documentation.



SUBMISSION 4)

What are John Doe's plans for March? Use GET /user_todo_by_month/ endpoint and take a screenshot of the request body in the interactive API documentation. (4pts)

7. Joining tables (2)

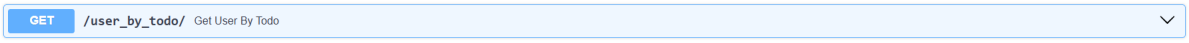
What if we only want to display specific columns?
Now we're going to retrieve a list of users who have a specific todo task.

```python
@app.get( path: "/user_by_todo/", status_code=status.HTTP_200_OK, tags=["User-Todo"])
async def get_user_by_todo(todo: str, db: db_dependency):
    db_users = (
        db.query(models.User.name,
                 models.Todo.due_year,
                 models.Todo.due_month,
                 models.Todo.due_day)
        .join(models.Todo)
        .filter(models.Todo.task_body.like(f"%{todo}%"))
        .all()
    )

    return [
        {
            "user_name": user.name,
            "due_year": user.due_year,
            "due_month": user.due_month,
            "due_day": user.due_day,
        }
        for user in db_users
    ]
```

Note that:

- Using .join(models.Todo) explicitly connects the User and Todo tables through user_id.
- .like() works the same way as the LIKE function in an SQL query.
- To return the results in JSON format:
    - Convert the db_users data retrieved with .all() using list comprehension.
    - FastAPI expects a list of dictionaries to properly format the response as JSON.

Now, you will see the following endpoint in the interactive API documentation.



SUBMISSION 5)

Who is going to Jamie's birthday party? You can simply enter 'birthday' as the parameter in the GET /user_by_todo/ endpoint and take a screenshot of the request body in the interactive API documentation. (4pts)